

Dynamic Protocol Priority Learning Engine (DPPLE)

1. Core Architecture: Adaptive Resource-Aware Protocol Management

A. Hardware-Aware Protocol Scheduler

python

```
class ProtocolResourceManager:
    def __init__(self):
        self.protocol_registry = {}
        self.resource_models = {}
        self.performance_history = []
        self.active_mode = "normal" # crisis, efficiency, diagnostic

    def register_protocol(self, protocol):
        """Register a protocol with its resource profile"""
        self.protocol_registry[protocol['name']] = {
            'base_priority': protocol.get('base_priority', 50),
            'resource_profile': self.measure_resource_profile(protocol),
            'minimum_requirements': protocol.get('minimum_requirements', {}),
            'can_defer': protocol.get('can_defer', True),
            'can_scale_down': protocol.get('can_scale_down', True),
            'scaling_options': protocol.get('scaling_options', {}),
            'performance_metrics': {
                'avg_cpu_usage': 0,
                'avg_memory_mb': 0,
                'avg_processing_ms': 0,
                'completion_rate': 1.0
            },
            'criticality_contexts': protocol.get('criticality_contexts', {})
        }

    def measure_resource_profile(self, protocol):
        """
        Dynamically measure protocol resource consumption
        """
        return {
            'cpu_percent': self.benchmark_cpu_usage(protocol),
            'memory_mb': self.benchmark_memory_usage(protocol),
            'disk_io_kb': self.benchmark_disk_usage(protocol),
            'npu_utilization': self.benchmark_npu_usage(protocol),
            'thermal_impact': self.estimate_thermal_impact(protocol),
            'energy_joules': self.estimate_energy_consumption(protocol)
        }
```

B. Real-Time Hardware State Monitor

python

```
class RPi5HardwareMonitor:
    def __init__(self):
        self.metrics = {
            'cpu': {'usage': 0, 'temp': 0, 'frequency': 0, 'throttled': False},
```

```

        'memory': {'used_mb': 0, 'available_mb': 0, 'swap_used_mb': 0},
        'thermal': {'core_temp': 0, 'throttle_history': [], 'fan_speed': 0},
        'power': {'voltage': 0, 'current_ma': 0, 'battery_percent': 0},
        'npu': {'utilization': 0, 'temp': 0, 'throughput': 0}
    }
    self.thresholds = self.load_thresholds()
    self.performance_baseline = self.establish_baseline()

def get_current_state(self):
    """Comprehensive hardware state assessment"""
    return {
        'timestamp': time.time(),
        'cpu_load': psutil.cpu_percent(interval=0.1, percpu=True),
        'cpu_temp': self.read_cpu_temp(),
        'memory': psutil.virtual_memory(),
        'swap': psutil.swap_memory(),
        'disk_io': psutil.disk_io_counters(),
        'network_io': psutil.net_io_counters(),
        'throttle_status': self.read_throttle_status(),
        'power_status': self.read_power_status()
    }

def assess_available_capacity(self):
    """
    Calculate available processing capacity in weighted units
    """
    current = self.get_current_state()

    # Calculate capacity scores (0-1)
    cpu_capacity = max(0, 1 - (np.mean(current['cpu_load']) / 100))
    thermal_capacity = max(0, 1 - (current['cpu_temp'] / 75)) # 75°C is
throttle limit
    memory_capacity = current['memory'].available / current['memory'].total

    # Weight the capacities
    total_capacity = (
        cpu_capacity * 0.4 +
        thermal_capacity * 0.3 +
        memory_capacity * 0.3
    )

    return {
        'total_capacity': total_capacity,
        'components': {
            'cpu': cpu_capacity,
            'thermal': thermal_capacity,
            'memory': memory_capacity
        },
        'state': self.classify_state(total_capacity)
    }

def classify_state(self, capacity):
    """Classify system state for protocol adjustment"""
    if capacity > 0.7:
        return "high_capacity"
    elif capacity > 0.4:
        return "normal_capacity"
    elif capacity > 0.2:
        return "reduced_capacity"
    elif capacity > 0.05:
        return "critical_capacity"
    else:
        return "overload"

```

2. Dynamic Priority Calculation Engine

A. Context-Aware Priority Matrix

python

```
class PriorityMatrixEngine:
    def __init__(self):
        self.context_weights = self.load_context_weights()
        self.protocol_relationships = self.map_protocol_dependencies()
        self.historical_effectiveness = {}

    def calculate_dynamic_priority(self, protocol_name, context):
        """
        Calculate real-time priority based on multiple factors
        """
        base_priority = self.protocol_registry[protocol_name]['base_priority']

        # Adjust based on current system state
        system_factor = self.calculate_system_factor(
            self.hardware_monitor.assess_available_capacity()
        )

        # Adjust based on context criticality
        context_factor = self.calculate_context_factor(
            context,
            protocol_name
        )

        # Adjust based on recent effectiveness
        effectiveness_factor = self.calculate_effectiveness_factor(
            protocol_name,
            context.get('user_id')
        )

        # Adjust based on protocol interdependencies
        dependency_factor = self.calculate_dependency_factor(
            protocol_name,
            context.get('active_protocols', [])
        )

        # Calculate final priority (0-100)
        final_priority = (
            base_priority * 0.3 +
            system_factor * 20 * 0.2 +
            context_factor * 100 * 0.3 +
            effectiveness_factor * 100 * 0.1 +
            dependency_factor * 100 * 0.1
        )

        # Apply emergency overrides
        if context.get('emergency_situation'):
            final_priority = min(100, final_priority * 1.5)

        return {
            'priority': min(100, max(0, final_priority)),
            'components': {
                'base': base_priority,
                'system': system_factor,
                'context': context_factor,
                'effectiveness': effectiveness_factor,
                'dependencies': dependency_factor
            },
        },
```

```

        'recommended_action': self.determine_action(final_priority)
    }

def calculate_system_factor(self, capacity_assessment):
    """
    Determine how much to prioritize protocols based on system state
    """
    state = capacity_assessment['state']

    # Different states prioritize different protocol types
    system_factors = {
        'high_capacity': {
            'monitoring_protocols': 0.8,
            'analysis_protocols': 0.9,
            'preventive_protocols': 0.7,
            'learning_protocols': 1.0 # Run learning when we have capacity
        },
        'normal_capacity': {
            'monitoring_protocols': 0.9,
            'analysis_protocols': 0.8,
            'preventive_protocols': 0.6,
            'learning_protocols': 0.5
        },
        'reduced_capacity': {
            'monitoring_protocols': 0.5,
            'analysis_protocols': 0.3,
            'preventive_protocols': 0.8, # Prevention is key when stressed
            'learning_protocols': 0.1 # Minimal learning
        },
        'critical_capacity': {
            'monitoring_protocols': 0.3,
            'analysis_protocols': 0.1,
            'preventive_protocols': 1.0, # Max prevention
            'learning_protocols': 0.0 # No learning
        },
        'overload': {
            'monitoring_protocols': 0.1,
            'analysis_protocols': 0.0,
            'preventive_protocols': 0.9, # Bare minimum prevention
            'learning_protocols': 0.0
        }
    }

    return system_factors[state]

def calculate_context_factor(self, context, protocol_name):
    """
    Determine protocol importance based on current situation
    """
    protocol = self.protocol_registry[protocol_name]
    context_criticality = protocol.get('criticality_contexts', {})

    # Check each context dimension
    factors = []

    # 1. User state context
    if context.get('user_state') in context_criticality.get('user_states', {}):
        factors.append(
            context_criticality['user_states'][context['user_state']]
        )

    # 2. Safety context
    if context.get('safety_concerns'):

```

```

        safety_factor = context_criticality.get('safety_impact', 0)
        factors.append(safety_factor * context['safety_concerns'])

# 3. Time context (time of day, duration, etc.)
time_factor = self.calculate_time_factor(context, protocol_name)
factors.append(time_factor)

# 4. Historical context (what's worked before)
historical_factor = self.get_historical_context_factor(
    context.get('user_id'),
    protocol_name,
    context.get('similar_situations', []))
)
factors.append(historical_factor)

return np.mean(factors) if factors else 0.5

```

B. Protocol Dependency & Conflict Mapping

python

```

class ProtocolRelationshipMapper:
    def __init__(self):
        self.dependency_graph = nx.DiGraph()
        self.conflict_matrix = {}
        self.synergy_scores = {}

    def map_protocol_relationships(self):
        """
        Create relationship maps between protocols
        """
        # Example mappings
        relationships = {
            'neurological_assessment': {
                'depends_on': ['somatic_monitoring'],
                'conflicts_with': ['deep_learning_analysis'], # Both CPU-
intensive
                'synergizes_with': ['behavioral_analysis', 'age_approximation'],
                'enhances': ['crisis_detection'],
                'reduces_need_for': ['generic_monitoring']
            },
            'crisis_intervention': {
                'depends_on': ['neurological_assessment',
'behavioral_analysis'],
                'supersedes': ['all_other_protocols'], # Emergency override
                'requires_full_resources': True
            }
        }

        # Build graph
        for protocol, rels in relationships.items():
            self.dependency_graph.add_node(protocol)

            for dep in rels.get('depends_on', []):
                self.dependency_graph.add_edge(dep, protocol, weight=1.0)

            for conflict in rels.get('conflicts_with', []):
                self.conflict_matrix.setdefault(protocol, {})[conflict] = -1.0
                self.conflict_matrix.setdefault(conflict, {})[protocol] = -1.0

            for synergy in rels.get('synergizes_with', []):
                self.synergy_scores.setdefault(protocol, {})[synergy] = 0.7

```

```

def calculate_dependency_factor(self, protocol_name, active_protocols):
    """
    Calculate how much this protocol depends on/conflicts with currently
    running protocols
    """
    if not active_protocols:
        return 0.5 # Neutral

    scores = []

    for active in active_protocols:
        # Check for conflicts
        conflict_score = self.conflict_matrix.get(protocol_name,
        {}).get(active, 0)
        if conflict_score < 0:
            scores.append(conflict_score) # Negative impact

        # Check for synergies
        synergy_score = self.synergy_scores.get(protocol_name,
        {}).get(active, 0)
        if synergy_score > 0:
            scores.append(synergy_score) # Positive impact

        # Check dependencies
        if protocol_name in self.dependency_graph:
            if active in nx.ancestors(self.dependency_graph, protocol_name):
                scores.append(0.8) # Dependency satisfied
            elif protocol_name in nx.ancestors(self.dependency_graph,
active):
                scores.append(0.3) # This enables others

    return np.mean(scores) if scores else 0.5

```

3. Adaptive Learning System

A. Outcome-Based Protocol Effectiveness Learning

python

```

class ProtocolEffectivenessLearner:
    def __init__(self):
        self.outcome_database = {}
        self.protocol_performance_history = {}
        self.adaptation_rules = self.load_adaptation_rules()

    def record_protocol_outcome(self, protocol_name, outcome_data):
        """
        Record the outcome of running a protocol
        """
        entry = {
            'timestamp': time.time(),
            'protocol': protocol_name,
            'resource_usage': outcome_data['resource_usage'],
            'effectiveness_score': outcome_data['effectiveness_score'],
            'context': outcome_data['context'],
            'system_state': outcome_data['system_state'],
            'user_response': outcome_data.get('user_response'),
            'alternative_protocols_considered': outcome_data.get('alternatives',
[])
        }

```

```

# Store outcome
self.outcome_database.setdefault(protocol_name, []).append(entry)

# Update performance metrics
self.update_performance_metrics(protocol_name, entry)

# Check if adaptation is needed
adaptation = self.check_for_adaptation_needed(protocol_name)

return adaptation

def calculate_effectiveness_score(self, protocol_name, outcome_data):
    """
    Calculate how effective the protocol was in this context
    """
    components = []

    # 1. Resource efficiency score
    resource_score = self.calculate_resource_efficiency(
        outcome_data['resource_usage'],
        outcome_data['system_state']['capacity']
    )
    components.append(('resource_efficiency', resource_score, 0.3))

    # 2. Outcome success score
    success_score = self.calculate_outcome_success(
        outcome_data['context'],
        outcome_data.get('user_response', {})
    )
    components.append(('outcome_success', success_score, 0.4))

    # 3. System impact score
    system_score = self.calculate_system_impact(
        outcome_data['system_state'],
        outcome_data['resource_usage']
    )
    components.append(('system_impact', system_score, 0.2))

    # 4. Alternative comparison score
    alternative_score = self.compare_with_alternatives(
        protocol_name,
        outcome_data['context'],
        outcome_data.get('alternative_protocols_considered', [])
    )
    components.append(('alternative_comparison', alternative_score, 0.1))

    # Calculate weighted effectiveness
    effectiveness = sum(score * weight for _, score, weight in components)

    return {
        'overall': effectiveness,
        'components': {name: score for name, score, _ in components}
    }

def update_performance_metrics(self, protocol_name, outcome_entry):
    """
    Update historical performance data for this protocol
    """
    if protocol_name not in self.protocol_performance_history:
        self.protocol_performance_history[protocol_name] = {
            'effectiveness_trend': [],
            'resource_usage_trend': [],
            'context_success_rates': {},
            'failure_patterns': [],

```

```

        'optimal_conditions': []
    }

    history = self.protocol_performance_history[protocol_name]

    # Update effectiveness trend
    history['effectiveness_trend'].append({
        'timestamp': outcome_entry['timestamp'],
        'score': outcome_entry['effectiveness_score']['overall'],
        'context': outcome_entry['context'].get('situation_type', 'unknown')
    })

    # Update context success rates
    context_key = self.create_context_key(outcome_entry['context'])
    if context_key not in history['context_success_rates']:
        history['context_success_rates'][context_key] = {
            'successes': 0,
            'attempts': 0,
            'avg_effectiveness': 0
        }

    ctx = history['context_success_rates'][context_key]
    ctx['attempts'] += 1
    if outcome_entry['effectiveness_score']['overall'] > 0.7:
        ctx['successes'] += 1
    ctx['avg_effectiveness'] = (
        (ctx['avg_effectiveness'] * (ctx['attempts'] - 1) +
         outcome_entry['effectiveness_score']['overall']) /
        ctx['attempts']
    )

    # Store optimal conditions if highly effective
    if outcome_entry['effectiveness_score']['overall'] > 0.85:
        history['optimal_conditions'].append({
            'context': outcome_entry['context'],
            'system_state': outcome_entry['system_state'],
            'effectiveness': outcome_entry['effectiveness_score']
        })

    # Prune old data to prevent memory issues
    self.prune_old_data(history)

```

B. Pattern Recognition for Optimal Protocol Selection

python

```

class ProtocolPatternRecognizer:
    def __init__(self):
        self.pattern_database = {}
        self.cluster_models = {}
        self.recommendation_engine = {}

    def analyze_success_patterns(self, protocol_history):
        """
        Identify patterns in successful protocol executions
        """
        patterns = {
            'context_patterns': {},
            'system_state_patterns': {},
            'temporal_patterns': {},
            'sequence_patterns': {}
        }

```



```

# Cluster successful executions
successful = [h for h in protocol_history
               if h['effectiveness_score']['overall'] > 0.8]

if len(successful) < 5:
    return patterns # Not enough data

# Analyze context patterns
context_features = self.extract_context_features(successful)
context_clusters = self.cluster_contexts(context_features)

patterns['context_patterns'] = {
    'clusters': context_clusters,
    'common_features': self.find_common_features(context_clusters)
}

# Analyze system state patterns
system_states = [h['system_state'] for h in successful]
optimal_states = self.find_optimal_system_states(system_states)

patterns['system_state_patterns'] = {
    'optimal_ranges': optimal_states,
    'avoid_states': self.find_avoid_states(protocol_history)
}

# Analyze temporal patterns (time of day, day of week, etc.)
temporal_features = self.extract_temporal_features(successful)
if temporal_features:
    patterns['temporal_patterns'] = self.identify_temporal_patterns(
        temporal_features
    )

# Analyze sequence patterns (which protocols work well together)
sequences = self.extract_protocol_sequences(protocol_history)
patterns['sequence_patterns'] =
self.identify_effective_sequences(sequences)

return patterns

def recommend_optimal_protocol(self, current_context, system_state):
    """
    Recommend the best protocol(s) for current conditions
    """
    recommendations = []

    # Check for exact pattern matches
    exact_match = self.find_exact_pattern_match(current_context,
system_state)
    if exact_match:
        recommendations.append({
            'protocol': exact_match['protocol'],
            'confidence': 0.95,
            'match_type': 'exact_pattern',
            'expected_effectiveness': exact_match['avg_effectiveness']
        })

    # Check for similar contexts
    similar_contexts = self.find_similar_contexts(current_context)
    for similar in similar_contexts:
        effectiveness = self.predict_effectiveness(
            similar['protocol'],
            current_context,
            system_state
        )

```

```

        if effectiveness['predicted'] > 0.7:
            recommendations.append({
                'protocol': similar['protocol'],
                'confidence': effectiveness['confidence'],
                'match_type': 'similar_context',
                'expected_effectiveness': effectiveness['predicted']
            })

    # Check for system-optimal protocols
    system_optimal = self.find_system_optimal_protocols(system_state)
    for protocol in system_optimal:
        recommendations.append({
            'protocol': protocol['name'],
            'confidence': protocol['fit_score'],
            'match_type': 'system_optimized',
            'expected_effectiveness': protocol['expected_effectiveness']
        })

    # Sort by expected effectiveness
    recommendations.sort(key=lambda x: x['expected_effectiveness'],
reverse=True)

    return recommendations

```

4. Dynamic Protocol Scaling & Execution

A. Protocol Scalability Manager

python

```

class ProtocolScalabilityManager:
    def __init__(self):
        self.scaling_profiles = {}
        self.quality_metrics = {}

    def create_scaling_profile(self, protocol_name, base_profile):
        """
        Create multiple scaling levels for a protocol
        """
        self.scaling_profiles[protocol_name] = {
            'full': {
                'name': 'full',
                'description': 'Complete protocol with all features',
                'resource_multiplier': 1.0,
                'quality_score': 1.0,
                'features': base_profile['features']
            },
            'balanced': {
                'name': 'balanced',
                'description': 'Optimized balance of accuracy and speed',
                'resource_multiplier': 0.65,
                'quality_score': 0.85,
                'features':
self.select_critical_features(base_profile['features'])
            },
            'light': {
                'name': 'light',
                'description': 'Minimal version for constrained resources',
                'resource_multiplier': 0.35,
                'quality_score': 0.65,

```

```

        'features':
self.select_minimal_features(base_profile['features'])
    },
    'emergency': {
        'name': 'emergency',
        'description': 'Emergency mode - only critical safety checks',
        'resource_multiplier': 0.15,
        'quality_score': 0.4,
        'features':
self.select_emergency_features(base_profile['features'])
    }
}

def select_optimal_scale(self, protocol_name, available_capacity, context):
    """
    Choose the best scaling level for current conditions
    """
    profiles = self.scaling_profiles[protocol_name]

    # Calculate capacity requirements for each profile
    requirements = {}
    for name, profile in profiles.items():
        # Estimate resource needs
        estimated_needs = self.estimate_resource_needs(profile, context)

        # Check if system can handle it
        can_run = self.can_system_handle(estimated_needs,
available_capacity)

        requirements[name] = {
            'profile': profile,
            'estimated_needs': estimated_needs,
            'can_run': can_run,
            'fitness_score': self.calculate_fitness_score(
                profile, context, available_capacity
            )
        }

    # Select best profile that can run
    runnable = {k: v for k, v in requirements.items() if v['can_run']}
    if not runnable:
        return None # Cannot run any version

    # Choose highest fitness score
    best_profile = max(runnable.items(), key=lambda x: x[1]
['fitness_score'])

    return {
        'selected_profile': best_profile[0],
        'profile_details': best_profile[1]['profile'],
        'fitness_score': best_profile[1]['fitness_score'],
        'estimated_resource_usage': best_profile[1]['estimated_needs']
    }

def calculate_fitness_score(self, profile, context, available_capacity):
    """
    Calculate how well this scaling level fits current situation
    """
    scores = []

    # Quality vs speed tradeoff based on context
    if context.get('requires_high_accuracy'):
        quality_weight = 0.7
        speed_weight = 0.3

```

```

elif context.get('requires_quick_response'):
    quality_weight = 0.3
    speed_weight = 0.7
else:
    quality_weight = 0.5
    speed_weight = 0.5

# Quality score
quality_score = profile['quality_score'] * quality_weight
scores.append(quality_score)

# Speed/efficiency score
efficiency_score = (1 - profile['resource_multiplier']) * speed_weight
scores.append(efficiency_score)

# System fit score
system_fit = min(1.0, available_capacity /
profile['resource_multiplier'])
scores.append(system_fit * 0.3)

# Context appropriateness
context_fit = self.assess_context_fit(profile, context)
scores.append(context_fit * 0.4)

return sum(scores) / len(scores)

```

B. Protocol Execution Orchestrator

python

```

class ProtocolOrchestrator:
    def __init__(self):
        self.resource_manager = ProtocolResourceManager()
        self.priority_engine = PriorityMatrixEngine()
        self.hardware_monitor = RPi5HardwareMonitor()
        self.scaling_manager = ProtocolScalabilityManager()
        self.learning_engine = ProtocolEffectivenessLearner()

        self.execution_queue = []
        self.active_protocols = []
        self.deferred_protocols = []

    def process_situation(self, user_input, context):
        """
        Main orchestration method
        """
        # 1. Assess current hardware state
        hardware_state = self.hardware_monitor.get_current_state()
        capacity = self.hardware_monitor.assess_available_capacity()

        # 2. Determine which protocols are needed
        needed_protocols = self.identify_needed_protocols(user_input, context)

        # 3. Calculate dynamic priorities
        prioritized_protocols = []
        for protocol in needed_protocols:
            priority_data = self.priority_engine.calculate_dynamic_priority(
                protocol['name'],
                **context, 'hardware_state': hardware_state
            )

            prioritized_protocols.append({
                **protocol,

```

```

        'dynamic_priority': priority_data['priority'],
        'priority_components': priority_data['components']
    })

    # 4. Sort by priority
    prioritized_protocols.sort(key=lambda x: x['dynamic_priority'],
reverse=True)

    # 5. Select which to run based on capacity
    protocols_to_run = []
    protocols_to_defer = []

    current_load = 0
    max_load = capacity['total_capacity']

    for protocol in prioritized_protocols:
        # Select scaling level
        scaling_decision = self.scaling_manager.select_optimal_scale(
            protocol['name'],
            max_load - current_load,
            context
        )

        if scaling_decision and (current_load +
scaling_decision['estimated_resource_usage']) <= max_load:
            protocols_to_run.append({
                **protocol,
                'scaling_level': scaling_decision['selected_profile'],
                'estimated_resources':
scaling_decision['estimated_resource_usage']
            })
            current_load += scaling_decision['estimated_resource_usage']
        else:
            protocols_to_defer.append(protocol)

    # 6. Execute protocols
    execution_results = []
    for protocol in protocols_to_run:
        result = self.execute_protocol(protocol, context)
        execution_results.append(result)

    # Record outcome for learning
    self.learning_engine.record_protocol_outcome(
        protocol['name'],
        {
            'resource_usage': self.measure_actual_usage(protocol),
            'effectiveness_score':
self.calculate_execution_effectiveness(result),
            'context': context,
            'system_state': hardware_state,
            'scaling_level': protocol['scaling_level']
        }
    )

    # 7. Handle deferred protocols
    if protocols_to_defer:
        self.handle_deferred_protocols(protocols_to_defer, context)

    # 8. Generate integrated response
    integrated_response = self.integrate_protocol_results(execution_results)

    # 9. Update system learning
    self.update_system_learning(execution_results, context)

```

```

    return {
        'executed_protocols': protocols_to_run,
        'deferred_protocols': protocols_to_defer,
        'hardware_state': hardware_state,
        'capacity_used': current_load,
        'capacity_available': max_load - current_load,
        'integrated_response': integrated_response,
        'learning_updates': self.get_learning_updates()
    }

def execute_protocol(self, protocol, context):
    """
    Execute a protocol at the selected scaling level
    """
    try:
        # Load the protocol implementation
        protocol_module = self.load_protocol_module(protocol['name'])

        # Apply scaling adjustments
        scaled_implementation = self.apply_scaling(
            protocol_module,
            protocol['scaling_level']
        )

        # Execute with resource limits
        with ResourceLimiter(protocol['estimated_resources']):
            result = scaled_implementation.execute(context)

        # Measure actual resource usage
        actual_usage = self.measure_resource_usage()

        return {
            'protocol': protocol['name'],
            'scaling_level': protocol['scaling_level'],
            'result': result,
            'resource_usage': actual_usage,
            'execution_time': time.time() - start_time,
            'success': True
        }

    except ResourceLimitExceeded:
        # Protocol exceeded its allocation
        return {
            'protocol': protocol['name'],
            'error': 'resource_limit_exceeded',
            'partial_result': self.get_partial_result_if_any(),
            'recommendation': 'increase_scaling_or_defer'
        }

    except Exception as e:
        # Other execution error
        return {
            'protocol': protocol['name'],
            'error': str(e),
            'success': False
        }

def handle_deferred_protocols(self, deferred_protocols, context):
    """
    Handle protocols that couldn't run due to resource constraints
    """
    # Store for later execution if capacity frees up
    self.deferred_protocols.extend([
        {
            **protocol,

```

```

        'deferral_time': time.time(),
        'context_snapshot': context,
        'priority_at_deferral': protocol['dynamic_priority']
    }
    for protocol in deferred_protocols
])

# Sort deferred by priority
self.deferred_protocols.sort(
    key=lambda x: x['priority_at_deferral'],
    reverse=True
)

# Prune old deferred protocols
current_time = time.time()
self.deferred_protocols = [
    p for p in self.deferred_protocols
    if current_time - p['deferral_time'] < 300 # 5 minutes max
]

# Log deferral for learning
self.log_deferral_event(deferred_protocols, context)

def check_deferred_execution(self):
    """
    Check if we can now run any deferred protocols
    """
    if not self.deferred_protocols:
        return []

    current_capacity = self.hardware_monitor.assess_available_capacity()

    # Only run deferred if we have spare capacity
    if current_capacity['state'] in ['high_capacity', 'normal_capacity']:
        # Try to run highest priority deferred protocols
        executed = []
        remaining_capacity = current_capacity['total_capacity']

        for protocol in list(self.deferred_protocols):
            scaling_decision = self.scaling_manager.select_optimal_scale(
                protocol['name'],
                remaining_capacity,
                protocol['context_snapshot']
            )

            if scaling_decision:
                # Execute deferred protocol
                result = self.execute_protocol({
                    **protocol,
                    'scaling_level': scaling_decision['selected_profile']
                }, protocol['context_snapshot'])

                executed.append(result)
                self.deferred_protocols.remove(protocol)

            # Update capacity
            remaining_capacity -=
scaling_decision['estimated_resource_usage']

            if remaining_capacity < 0.1: # Leave some buffer
                break

        return executed

```

```
return []
```

5. Learning & Adaptation System

A. Continuous Optimization Engine

python

```
class ContinuousOptimizationEngine:
    def __init__(self):
        self.optimization_history = []
        self.parameter_space = self.define_parameter_space()
        self.bayesian_optimizer = BayesianOptimizer()
        self.genetic_algorithm = GeneticOptimizer()

    def define_parameter_space(self):
        """
        Define parameters that can be optimized
        """
        return {
            'protocol_priorities': {
                'type': 'continuous',
                'bounds': [0, 100],
                'protocols': list(self.protocol_registry.keys())
            },
            'scaling_thresholds': {
                'type': 'continuous',
                'bounds': [0, 1],
                'parameters': ['high_capacity_threshold',
'normal_capacity_threshold']
            },
            'resource_allocation_strategy': {
                'type': 'categorical',
                'options': ['conservative', 'balanced', 'aggressive',
'adaptive']
            },
            'deferral_policy': {
                'type': 'categorical',
                'options': ['strict', 'flexible', 'context_aware',
'learning_based']
            }
        }

    def run_optimization_cycle(self, performance_data):
        """
        Run optimization to improve protocol management
        """
        # 1. Analyze current performance
        current_performance = self.analyze_performance(performance_data)

        # 2. Identify optimization opportunities
        opportunities =
self.identify_optimization_opportunities(current_performance)

        # 3. Generate optimization hypotheses
        hypotheses = self.generate_optimization_hypotheses(opportunities)

        # 4. Test hypotheses (simulated or small-scale)
        test_results = self.test_hypotheses(hypotheses)

        # 5. Select best improvements
```



```

improvements = self.select_best_improvements(test_results)

# 6. Apply improvements
self.apply_improvements(improvements)

# 7. Monitor results
self.monitor_improvement_impact(improvements)

return {
    'optimization_cycle': len(self.optimization_history) + 1,
    'opportunities_found': len(opportunities),
    'hypotheses_tested': len(test_results),
    'improvements_applied': len(improvements),
    'expected_improvement':
self.calculate_expected_improvement(improvements)
}

def identify_optimization_opportunities(self, performance_data):
    """
    Find areas where the system could be improved
    """
    opportunities = []

    # Look for protocols with consistently poor effectiveness
    for protocol, data in
performance_data['protocol_effectiveness'].items():
        if data['avg_effectiveness'] < 0.6:
            opportunities.append({
                'type': 'low_effectiveness',
                'protocol': protocol,
                'avg_score': data['avg_effectiveness'],
                'suggested_action': 'adjust_priority_or_scaling'
            })

    # Look for resource bottlenecks
    resource_bottlenecks = self.identify_resource_bottlenecks(
        performance_data['resource_usage']
    )
    opportunities.extend(resource_bottlenecks)

    # Look for protocols that are frequently deferred
    deferral_analysis = self.analyze_deferral_patterns(
        performance_data['deferral_log']
    )
    opportunities.extend(deferral_analysis)

    # Look for context mismatches
    context_mismatches = self.find_context_mismatches(
        performance_data['context_success_rates']
    )
    opportunities.extend(context_mismatches)

    return opportunities

def apply_improvements(self, improvements):
    """
    Apply the selected improvements to the system
    """
    applied = []

    for improvement in improvements:
        try:
            if improvement['type'] == 'protocol_priority_adjustment':
                # Update protocol priority

```

```

        self.protocol_registry[improvement['protocol']]
['base_priority'] = \
        improvement['new_priority']
        applied.append(improvement)

    elif improvement['type'] == 'scaling_adjustment':
        # Update scaling profile
        self.scaling_manager.update_scaling_profile(
            improvement['protocol'],
            improvement['new_profile']
        )
        applied.append(improvement)

    elif improvement['type'] == 'resource_allocation_adjustment':
        # Update allocation strategy
        self.resource_manager.set_allocation_strategy(
            improvement['new_strategy']
        )
        applied.append(improvement)

    elif improvement['type'] == 'deferral_policy_adjustment':
        # Update deferral policy
        self.orchestrator.set_deferral_policy(
            improvement['new_policy']
        )
        applied.append(improvement)

    # Log the improvement
    self.log_improvement(improvement)

except Exception as e:
    self.log_improvement_error(improvement, e)

return applied

```

B. Self-Monitoring & Health Assessment

python

```

class SystemHealthMonitor:
    def __init__(self):
        self.health_metrics = {}
        self.performance_trends = {}
        self.anomaly_detector = AnomalyDetector()

    def assess_system_health(self):
        """
        Comprehensive health assessment of the protocol management system
        """
        health_report = {
            'timestamp': time.time(),
            'overall_health': 0,
            'components': {},
            'recommendations': [],
            'anomalies': []
        }

        # 1. Hardware health
        hardware_health = self.assess_hardware_health()
        health_report['components']['hardware'] = hardware_health

        # 2. Protocol execution health
        protocol_health = self.assess_protocol_health()

```

```

health_report['components']['protocols'] = protocol_health

# 3. Learning system health
learning_health = self.assess_learning_health()
health_report['components']['learning'] = learning_health

# 4. Resource management health
resource_health = self.assess_resource_management_health()
health_report['components']['resource_management'] = resource_health

# 5. Calculate overall health
component_scores = [c['score'] for c in
health_report['components'].values()]
health_report['overall_health'] = np.mean(component_scores)

# 6. Detect anomalies
anomalies = self.anomaly_detector.detect_anomalies(health_report)
health_report['anomalies'] = anomalies

# 7. Generate recommendations
recommendations = self.generate_health_recommendations(health_report)
health_report['recommendations'] = recommendations

# 8. Store health history
self.store_health_history(health_report)

return health_report

def generate_health_recommendations(self, health_report):
    """
    Generate specific recommendations to improve system health
    """
    recommendations = []

    # Check hardware health
    if health_report['components']['hardware']['score'] < 0.7:
        recommendations.append({
            'priority': 'high',
            'action': 'hardware_maintenance',
            'description': 'Hardware showing signs of stress. Consider
cooling improvements or load reduction.',
            'specifics': health_report['components']['hardware']['issues']
        })

    # Check protocol health
    low_performance_protocols = [
        p for p, data in health_report['components']['protocols']
['protocol_details'].items()
        if data['effectiveness'] < 0.6
    ]

    if low_performance_protocols:
        recommendations.append({
            'priority': 'medium',
            'action': 'protocol_optimization',
            'description': f'{len(low_performance_protocols)} protocols
showing low effectiveness.',
            'protocols': low_performance_protocols,
            'suggested_actions': [
                'Review scaling profiles',
                'Adjust priority weights',
                'Consider alternative implementations'
            ]
        })

```

```

# Check learning health
if health_report['components']['learning']['score'] < 0.5:
    recommendations.append({
        'priority': 'medium',
        'action': 'learning_system_maintenance',
        'description': 'Learning system not converging effectively.',
        'suggested_actions': [
            'Reset learning rates',
            'Increase exploration',
            'Review reward function'
        ]
    })

return recommendations

```

6. Implementation Strategy

A. Phased Implementation Plan

Phase 1: Foundation (Weeks 1-2)

yaml

Objectives:

- Implement basic hardware monitoring
- Create protocol registry with static priorities
- Build simple capacity-based protocol selection
- Establish basic execution orchestration

Key Components:

- RPi5HardwareMonitor (basic)
- ProtocolResourceManager (static)
- SimpleProtocolOrchestrator

Phase 2: Dynamic Adaptation (Weeks 3-4)

yaml

Objectives:

- Implement dynamic priority calculation
- Add protocol scaling profiles
- Create basic learning system
- Add deferral handling

Key Components:

- PriorityMatrixEngine
- ProtocolScalabilityManager
- BasicProtocolEffectivenessLearner
- DeferredProtocolHandler

Phase 3: Advanced Learning (Weeks 5-6)

yaml

Objectives:

- Implement pattern recognition
- Add Bayesian optimization
- Create health monitoring
- Add anomaly detection

Key Components:

- ProtocolPatternRecognizer
- ContinuousOptimizationEngine
- SystemHealthMonitor
- AnomalyDetector

Phase 4: Integration & Refinement (Weeks 7-8)

yaml

Objectives:

- Full system integration
- Performance optimization for RPi5
- Real-world testing
- Iterative refinement

Key Components:

- Complete ProtocolOrchestrator
- Performance tuning
- Field testing framework
- Feedback integration

B. Configuration Files

protocol_configuration.json

json

```
{
  "protocol_registry": {
    "neurological_assessment": {
      "base_priority": 85,
      "minimum_requirements": {
        "cpu_percent": 15,
        "memory_mb": 50,
        "processing_time_ms": 200
      },
      "can_defer": true,
      "can_scale_down": true,
      "scaling_options": {
        "full": {"cpu": 20, "memory": 80, "accuracy": 0.95},
        "balanced": {"cpu": 12, "memory": 50, "accuracy": 0.85},
        "light": {"cpu": 7, "memory": 30, "accuracy": 0.70},
        "emergency": {"cpu": 3, "memory": 15, "accuracy": 0.50}
      },
      "criticality_contexts": {
        "user_states": {"crisis": 0.9, "distressed": 0.7, "normal": 0.5},
        "safety_impact": 0.8,
        "time_factors": {"night": 0.6, "weekend": 0.5}
      }
    }
  },
  "resource_management": {
    "capacity_thresholds": {
      "high_capacity": 0.7,
      "normal_capacity": 0.4,
      "reduced_capacity": 0.2,
      "critical_capacity": 0.05
    },
    "allocation_strategies": {
      "conservative": {"buffer": 0.3, "safety_margin": 0.2},
      "balanced": {"buffer": 0.2, "safety_margin": 0.1},
      "aggressive": {"buffer": 0.1, "safety_margin": 0.05},
    }
  }
}
```

```

    "adaptive": {"buffer": "dynamic", "safety_margin": "context_aware"}
  }
}

```

learning_configuration.json

json

```

{
  "learning_parameters": {
    "effectiveness_learning": {
      "min_samples_for_learning": 10,
      "learning_rate": 0.1,
      "forgetting_factor": 0.95,
      "exploration_rate": 0.1
    },
    "pattern_recognition": {
      "context_clustering_samples": 20,
      "temporal_pattern_min_occurrences": 5,
      "sequence_pattern_min_length": 3
    },
    "optimization": {
      "optimization_cycle_hours": 24,
      "hypothesis_test_duration_minutes": 30,
      "improvement_evaluation_period_hours": 12
    }
  },
  "performance_targets": {
    "protocol_effectiveness": {
      "target_score": 0.8,
      "minimum_acceptable": 0.6,
      "exceptional_threshold": 0.9
    },
    "resource_efficiency": {
      "cpu_utilization_target": 0.7,
      "memory_utilization_target": 0.75,
      "thermal_safety_margin": 10
    },
    "response_times": {
      "initial_response_max_ms": 1000,
      "protocol_execution_max_ms": 500,
      "learning_update_max_ms": 100
    }
  }
}

```

7. Key Benefits & Innovations

A. System Advantages

1. **Hardware-Aware Optimization:** Every decision considers current RPi5 state
2. **Dynamic Priority Calculation:** Priorities change based on context, not just static rules
3. **Adaptive Scaling:** Protocols automatically adjust to available resources
4. **Continuous Learning:** System improves with every interaction
5. **Graceful Degradation:** Maintains core functions even under heavy load

6. **Self-Healing:** Detects and corrects performance issues automatically

B. RPi5-Specific Optimizations

1. **Thermal Management:** Protocols adjust based on CPU temperature
2. **NPU Utilization:** Leverages AI accelerator when available
3. **Memory-Aware Execution:** Prevents swap usage as configured
4. **Power-Aware Scheduling:** Optimizes for battery life when applicable
5. **Real-Time Adaptation:** Responds to changing hardware conditions within seconds

C. Learning Capabilities

1. **Protocol Effectiveness Learning:** Learns which protocols work best in which contexts
2. **Resource Usage Prediction:** Improves resource estimation accuracy over time
3. **Pattern Recognition:** Identifies successful protocol combinations
4. **Optimization Loop:** Continuously improves system parameters
5. **Anomaly Detection:** Identifies and addresses performance issues

Conclusion

The **Dynamic Protocol Priority Learning Engine (DPPLE)** transforms hardware limitations from a constraint into an **optimization opportunity**. By:

1. **Continuously monitoring** hardware state
2. **Dynamically calculating** protocol priorities
3. **Intelligently scaling** protocol execution
4. **Learning from every outcome**
5. **Self-optimizing** over time

We create a system that becomes **more efficient and effective** precisely when resources are constrained. The RPi5's limitations become the training ground for an exceptionally resource-aware AI system.

This approach embodies the resilience principle: "**What doesn't kill the system makes it smarter.**" Each resource-constrained situation teaches the system how to better allocate its limited resources, creating a positive feedback loop of increasing efficiency and effectiveness.

The system learns not just about user needs, but about **its own operational capabilities**—creating a truly self-aware AI that optimizes both its external protective functions and its internal resource management in harmony.